

Model Generation Scripts

ThreeDViewport ships with three Python scripts that generate every `.glb` file referenced from the included sample projects. They use `trimesh` for geometry and `PIL` for procedural texture maps, then export glTF 2.0 with embedded PBR materials.

Script	Purpose
<code>generate_models.py</code>	Build a single-mesh shape (cube, sphere, mobius, molecule, etc.) with a chosen colour palette and material preset. Interactive prompts.
<code>generate_character.py</code>	Build the rigged <code>robot</code> character with a 4-region colour scheme (body / head / arms / legs). Preserves the FK hierarchy needed for follow-camera and pose animation.
<code>generate_all.py</code>	Batch driver. Calls the other two to produce every colour × material combination of every shape (3,000 files), plus 30 uniform-colour robots — optionally plus 900 two-tone robots.

Setup

The scripts depend on `trimesh`, `pillow`, and `numpy`. They expect a venv at `/tmp/glb_env`. That path is ephemeral — recreate it after a reboot:

```
bash
python3 -m venv /tmp/glb_env
/tmp/glb_env/bin/pip install trimesh pillow numpy
```

All commands below are run via the venv's interpreter, e.g.:

```
bash
/tmp/glb_env/bin/python3 generate_all.py
```

You can also use any other Python that has the three packages installed.

Where the files go

Script	Output root
<code>generate_models.py</code>	<code>./Models/</code> (alongside the script, in the repo root)
<code>generate_character.py</code>	<code>./Models/</code> (alongside the script)
<code>generate_all.py</code>	<code>~/Documents/ThreeDViewport/Models/</code> (the same path the macOS app's project files reference)

`generate_all.py` writes shapes into subfolders by shape name (e.g. `~/Documents/ThreeDViewport/Models/sphere/`) and robots into `~/Documents/ThreeDViewport/Models/robot/` . The three `buckyball-*` shapes all land in a single `buckyball/` subfolder.

Shared colour and material palette

`generate_models.py` defines the colour palette and material list; `generate_character.py` inlines copies of the same tables so it stays runnable on its own. The two stay in sync by convention.

Greyscale tonal ranges (keys 1–6)

A greyscale texture is remapped into a sub-range. Useful when you want monochrome models that still react crisply to lighting.

Key	Name	Range
1	Dark	10 – 90
2	Dark-Mid	40 – 150
3	Mid	80 – 180
4	Mid-Bright	130 – 220
5	Bright	180 – 255
6	Wide	0 – 255 (full range)

Colour palettes (keys 7–14)

Each palette is a three-stop gradient (dark → mid → bright). Two complementary variants are available per palette:

Key	Name
7	Warm Amber
8	Cool Steel
9	Forest
10	Rose
11	Sunset
12	Ocean
13	Cosmic
14	Copper

Suffix	What it does
(none)	Plain three-stop gradient.
c1	Replaces the dark stop with a single complementary colour — e.g. 11c1 is Sunset with a teal pool of shadow.
c2	Inserts two complementary stops (at greyscale values 0 and 96) before the palette's mid stop — a richer dark-to-complement blend.

Append **a** to any choice to lock it in for the remaining prompts (e.g. 11c2a). A bare **a** skips all remaining prompts and uses defaults.

Material presets

Key	Name	Metallic	Roughness
p	Polished Metal	1.00	0.10
b	Brushed Metal	0.90	0.45
m	Matte Plastic	0.00	0.85
c	Ceramic	0.05	0.40
r	Rubber	0.00	0.95

m is the default when you press Enter.

Total combinations

generate_models.py

```
bash
/tmp/glb_env/bin/python3 generate_models.py      # interactive
/tmp/glb_env/bin/python3 generate_models.py -y    # accept all defaults (greyscale + i
```

Builds every shape in [SHAPES](#) (currently 20) one at a time, prompting for a colour palette and a material preset before each shape. Filenames encode the choice, e.g. [sphere-sunset-c2-ceramic.glb](#).

Shape catalog

Group	Shapes	Default texture
Primitives	cube · cylinder · pyramid · sphere · torus · tetrahedron · octahedron · hexprism · capsule	mostly linear / radial / cylindrical gradients
Surfaces	mobius · star · hyperboloid · trefoil · helix	spiral, cells, concentric rings, diagonal stripes, wood grain
Buckyballs	buckyball-162 · buckyball-642 · buckyball-2562	icospheres at three subdivisions (162, 642, 2562 verts)
Molecules	water · methane · benzene	radial gradient / marble / angular stripes

Each builder returns [\(mesh, uv, gray_array\)](#). The greyscale texture is coloured by the chosen palette function, encoded as a PNG, and embedded as the mesh's [baseColorTexture](#). UVs are precomputed per shape (box / cylindrical / spherical / per-vertex along path).

Molecule note

In [generate_models.py](#)'s standalone mode, molecules (water, methane, benzene) get a single PBR material like every other shape. In [generate_all.py](#), the [c1](#) and [c2](#) palette variants of molecules instead use [MOLECULE_BUILDERS](#) to produce a multi-part scene with three solid-colour groups — [heavy](#) (O / C atoms), [hydrogen](#) (H atoms), [bonds](#) (cylinders) — coloured from the palette's bright / complementary / mid stops. This makes molecule colourings read as chemistry rather than as a single textured blob.

generate_character.py

```
bash
/tmp/glb_env/bin/python3 generate_character.py      # interactive
/tmp/glb_env/bin/python3 generate_character.py -y  # default greyscale
```

Builds a rigged robot as `character-{bodyColor}-{headColor}-{armColor}-{legColor}.glb`. The four colour prompts are:

Region	Parts
Body	hips · torso · chest_panel · neck
Head	head sphere · visor · eyes · ears · antenna
Arms + Hands	shoulder · upper arm · elbow · forearm · hand (× left/right)
Legs + Feet	upper leg · knee · lower leg · ankle · foot (× left/right)

FK hierarchy

Every part's node origin sits at the joint that controls it, so rotating a node in ThreeDViewport pivots around the anatomical joint (important for keyframe animation and for camera-follow on sub-parts).

```
hips                                     (world root)
├─ torso                               pivot: hip-spine joint    [0, 0.145, 0]
│   ├─ chest_panel
│   └─ neck                           pivot: top of torso      [0, 0.60, 0]
│       └─ head                       pivot: top of neck      [0, 0.69, 0]
│           ├─ visor, eye_L, eye_R
│           ├─ ear_L, ear_R
│           └─ antenna_pole, antenna_ball
│   └─ shoulder_L/R                   pivot: shoulder joint    [±0.305, 0.565, 0]
│       └─ upper_arm_L/R               (same pivot as shoulder)
│           └─ elbow_L/R               pivot: elbow joint      [±0.305, 0.215, 0]
│               └─ forearm_L/R         (same pivot as elbow)
│                   └─ hand_L/R        pivot: wrist            [±0.305, -0.025, 0]
├─ upper_leg_L/R                       pivot: hip-leg joint    [±0.135, -0.025, 0]
│   └─ knee_L/R                       pivot: knee joint       [±0.135, -0.325, 0]
│       └─ lower_leg_L/R              (same pivot as knee)
│           └─ ankle_L/R              pivot: ankle joint     [±0.135, -0.625, 0]
│               └─ foot_L/R           (same pivot as ankle)
```

The script prints the final scene graph after building so you can verify the parenting matches.

Per-part materials

Different parts of the robot use different material presets internally:

Preset	Used by
<code>_METAL</code> (matte plastic)	hips, torso, shoulders, upper arms, forearms, upper legs, lower legs
<code>_HEAD</code> (rubbery sphere)	head
<code>_GLASS</code> (low-roughness with thin gloss)	chest panel, visor
<code>_JOINT</code> (slightly metallic)	neck, ears, antenna pole, elbow, hand, knee, ankle
<code>_GLOW(...)</code> (emissive)	eye LEDs, antenna ball

These are constants inside the script — adjust them there if you want a different look for the robot.

generate_all.py

```
bash
/tmp/glb_env/bin/python3 generate_all.py          # shapes + 30 uniform robots
/tmp/glb_env/bin/python3 generate_all.py --two-tone # also generates 900 two-tone robots
/tmp/glb_env/bin/python3 generate_all.py --shapes-only
/tmp/glb_env/bin/python3 generate_all.py --robot-only
```

Non-interactive. Iterates every (shape × colour × material) and every (robot × colour) combination, writing into `~/Documents/ThreeDViewport/Models/`. Progress is printed on a single rewriting line per shape so the output stays terse.

Mode	File count
Default (shapes + uniform robots)	$20 \times 30 \times 5 + 30 = 3,030$
<code>--shapes-only</code>	$20 \times 30 \times 5 = 3,000$
<code>--robot-only</code>	30
<code>--robot-only --two-tone</code>	$30 + 900 = 930$
Default + <code>--two-tone</code>	3,930

Two-tone robots use body+legs as colour A and head+arms as colour B, named `robot-{A}-{B}.glb`.

The shapes pass uses `palette_molecule_colors()` for molecule shapes when the palette is `c1` or `c2`, producing the multi-part atom/bond colour split described above.

Adding a new shape

1. Write a builder in `generate_models.py` that returns `(mesh, uv, gray_array)` — `mesh` is a `trimesh.Trimesh`, `uv` is per-vertex `(N, 2)`, `gray_array` is a 2D `uint8` greyscale texture (typically `TEX_SIZE × TEX_SIZE`).
2. Append it to `SHAPES` as `(name, builder, "human-readable pattern name")`.
3. If the output should live in a shared subfolder (like the buckyballs do), add an entry to `SHAPE_OUTPUT_DIRS`.
4. Run `generate_all.py` — it picks the new shape up automatically.

For a multi-coloured molecule with `c1` / `c2` variants:

1. Add a `build_X_scene(heavy_rgb, h_rgb, bond_rgb, metalness, roughness)` function that returns a `trimesh.Scene` with three nodes named `heavy`, `hydrogen`, `bonds` (each with a solid-colour PBR material).
2. Register it in the `MOLECULE_BUILDERS` dict, keyed by the shape name you used in `SHAPES`.
3. `generate_all.py` will route `c1` / `c2` palette variants through your scene builder while keeping plain palettes / greyscale on the single-mesh `builder()` path.